GenSpark

Manual Testing and Writing a Test Plan

You have written programs that check input, make decisions, and repeat. The next question is how you know they work. "I ran it once and it looked fine" is not an answer. This article teaches you to test a program on purpose: decide what it should do, try it in a planned way, and write down what happened. That written record is called a test plan. You will write one for the Number Guessing Game, and in Week 2 you will see how a tool called pytest automates the same idea.

Testing and Debugging Are Different Jobs

Testing is looking for problems. You run the program with chosen inputs and compare what happens to what should happen.

Debugging is fixing a problem once you have found it. You hunt for the cause and change the code.

Keep them separate in your head. Testing asks, "Does it work?" Debugging asks, "Why doesn't it?" A good tester tries to break the program. A test that finds a bug has done its job.

Anatomy of a Test Case

A test case is one specific check. It has four parts:

Part	Question it answers	Example
What I tested	Which behavior is this checking?	A guess that is too low
What I did	What exact steps or input?	Set `DEBUG_SECRET = 42`, then guess 10
Expected result	What should happen?	The game says "Too low"

Actual result	What did happen?	The game said "Too low"

You then mark the case **Pass** if actual matches expected, and **Fail** if not.

Write the expected result before you run it

This is the most important rule in testing. Decide what should happen **first**, then run the program. If you run it first and write down whatever it did, you will talk yourself into believing the output is right. Writing the expectation ahead of time keeps you honest.

Make every case repeatable

Someone else should be able to repeat your test and get the same result. "I typed some bad stuff" is not repeatable. "I typed `abc` and pressed Enter" is. Always write the exact input.

One idea per case

If a case checks three things, a failure does not tell you which one broke. Keep each case focused.

Where Test Ideas Come From

You do not have to invent tests out of thin air. Start from the requirements, the list of things the program must do. For each requirement, ask what could go wrong. Then cover these kinds of cases:

Kind	What it checks	Example
Normal	The usual, expected use	A guess of 30 when the secret is 42
Boundary	Values at the edges of a rule	The lowest and highest allowed numbers
Invalid input	Things the user should not type	Letters, a blank entry, a negative number
State and sequence	Something that changes over several steps	The attempt count after a bad entry
Time and randomness	Behavior that depends on the clock or chance	A timeout, a forced secret number

Boundaries are where bugs hide

Most bugs live at the edges of a rule. If children under 13 get a discount, the dangerous ages are 12 and 13, not 6. A mistake such as writing `<` instead of `<=` only shows up at the edge. This is the off by one error you met with `range()`.

For any rule, test three values: **just below** the edge, **on** the edge, and **just above** it. For a limit of 100, test 99, 100, and 101.

A Worked Example

Here is a small program that picks a ticket price from an age. Its requirements are:

1. Under 5 is free, 5 to 12 is a child ticket, 13 to 64 is an adult ticket, and 65 and over is a senior ticket.
2. Anything that is not a whole number is rejected with a message.
3. Ages above 120 are rejected with a message.

Suppose the first version of the program was written before requirement 3 was noticed:

python

```
age_text = input("Enter your age: ").strip()

if not age_text.isdigit():
    print("Please enter a whole number.")
else:
    age = int(age_text)
    if age < 5:
        print("Free")
    elif age <= 12:
        print("Child ticket")
    elif age <= 64:
        print("Adult ticket")
    else:
        print("Senior ticket")
```

Step 1: List the test cases and write the expected results

Before running anything, build the plan. Look at how the boundaries are covered:

#	What I tested	What I did	Expected result
1	Lowest age	Entered `0`	Free
2	Just below child	Entered `4`	Free
3	First child age	Entered `5`	Child ticket
4	Last child age	Entered `12`	Child ticket
5	First adult age	Entered `13`	Adult ticket
6	Last adult age	Entered `64`	Adult ticket
7	First senior age	Entered `65`	Senior ticket
8	Letters	Entered `abc`	Please enter a whole number.
9	Blank entry	Pressed Enter with nothing typed	Please enter a whole number.
10	Negative number	Entered `-3`	Please enter a whole number.
11	Decimal	Entered `3.5`	Please enter a whole number.
12	Extra spaces	Entered `30` (spaces around 30)	Adult ticket
13	Age above the maximum	Entered `200`	Please enter an age from 0 to 120.

Cases 1 to 7 are boundary cases. Cases 8 to 11 are invalid input. Case 12 checks that `strip()` is doing its job. Case 13 comes straight from requirement 3.

Step 2: Run each case and record the actual result

Run the program 13 times and fill in what happened. Cases 1 to 12 behave as expected. Case 13 does not:

#	Expected result	Actual result	Pass?
12	Adult ticket	Adult ticket	Yes
13	Please enter an age from 0 to 120.	Senior ticket	**No**

The test found a bug. The program has no upper limit, so 200 is happily treated as a senior.

Step 3: Record the failure, fix it, and retest

Do not erase the failed row. Write down what you found and how you fixed it. That record is evidence that your testing works. The fix adds a limit and a check:

python

```
MAX_AGE = 120

age_text = input("Enter your age: ").strip()

if not age_text.isdigit():
    print("Please enter a whole number.")
else:
    age = int(age_text)
    if age > MAX_AGE:
        print(f"Please enter an age from 0 to {MAX_AGE}.")
    elif age < 5:
        print("Free")
    elif age <= 12:
        print("Child ticket")
    elif age <= 64:
        print("Adult ticket")
    else:
        print("Senior ticket")
```

The check for a too-large age comes first, so a huge number never reaches the ticket choices. Run case 13 again. It now passes. Then add two boundary cases for the new rule:

#	What I tested	What I did	Expected result	Actual result	Pass?
13	Age above the maximum	Entered `200`	Please enter an age from 0 to 120.	First run: Senior ticket. After fix: Please enter an age from 0 to 120.	Yes (fixed)
14	Highest allowed age	Entered `120`	Senior ticket	Senior ticket	Yes
15	Just above the maximum	Entered `121`	Please enter an age from 0 to 120.	Please enter an age from 0 to 120.	Yes

Step 4: Rerun the old cases too

A fix can break something that used to work. After any change, rerun the **earlier** cases, not just the one you fixed. This is called regression testing. In the example, rerunning cases 1 to 12 confirms that adding the age limit did not disturb the ticket prices. In a bigger program, this is the reason testers dread changing code: every change means rerunning everything. That is exactly what automation solves.

Controlling Randomness and Time

Some behavior is hard to test because you cannot predict it. You solved both problems in the last article.

Randomness. A game that picks a random secret number gives you no expected result to write down. Use your `DEBUG_SECRET` setting to force a known number:

python

```
DEBUG_SECRET = 42      # for testing only. Set back to None before you commit.
```

Now a guess of 10 should say "Too low", a guess of 90 should say "Too high", and a guess of 42 should win. You can write the expected result ahead of time.

Time. Nobody wants to wait 60 seconds for every timer test. Shorten the limit:

python

```
TIME_LIMIT = 5        # for testing only. Set back to 60 before you commit.
```

Start the round, wait more than 5 seconds, then submit a guess. The timeout message should appear. To test the opposite case, a guess made quickly, answer right away and confirm the round continues.

Always restore these settings. A test setting left in your finished code ships a broken game. Make restoring `DEBUG_SECRET` and `TIME_LIMIT` part of your final checklist. A good habit is to write "reset test settings" in your test plan as the last row.

Testing State Over Several Steps

Some bugs only show up after several actions. The attempt counter is a good example. A bad entry should **not** change it, but a valid guess should. To test that, you need a sequence:

What I did	What I expect
Start a game with 8 attempts	"Attempts remaining" will start from 8
Enter `abc`	Message about a whole number. Attempts unchanged
Enter `30` (valid, wrong)	"Too low". Attempts remaining: 7
Enter nothing	Message about a whole number. Attempts still 7
Enter `35`	"Too low". Attempts remaining: 6

Write sequences like this as a single case, with the steps in order and the expected result for the final state. The count after the bad entries is what proves the rule.

A Shortcut: Feeding Inputs from the Terminal

Typing the same inputs again and again is slow. In the Mac Terminal, you can send a list of inputs to your program all at once. `printf` prints each input on its own line, and the `|` (pipe) hands them to your program as if you had typed them:

bash

```
printf 'abc\n\n0\n101\n30\n60\n42\n' | python3 guess.py
```

Each `\n` stands for pressing Enter, and `\n\n` is a blank entry. The program reads one line for every `input()` call. This is a fast way to repeat a test exactly.

Two warnings:

- If your program asks for more input than you supply, it stops with `EOFError: EOF when reading a line`. Add more lines, or check that your game actually needed that many.
- Timing tests do not work well this way, because the inputs arrive instantly. Test timeouts by hand.

This is optional. Typing by hand is fine, and for your first test plan it is worth doing a few cases manually to see the program the way a user does.

Writing Your Test Plan in the README

Your Number Guessing Game README must include a test plan. A README is written in Markdown, and GitHub turns Markdown tables into neat tables automatically. The syntax is simple:

markdown

```
| # | What I tested | What I did | Expected result | Actual result | Pass? |
|---|---------------|------------|-----------------|---------------|-------|
| 1 | Guess below the secret | Set DEBUG_SECRET to 42, guessed 10 | Says too l
| 2 | Letters as input | Entered abc | Message, attempts unchanged | Message,
```

The rules:

- Each row is one line, with `|` between columns and at both ends.
- The second line, made of dashes, is required. It separates the header from the rows.
- Columns do not need to line up in the file. GitHub fixes the spacing when it displays the table.
- Do not use a `|` character inside a cell. It splits the cell. Describe it in words instead.

- You cannot put a line break inside a cell. Keep each cell short, or use `<br>` if you must.
- To see your table the way a reader will, open the README on GitHub, or use the preview in PyCharm.

What makes a good plan

- **Cover every requirement.** Go through the lab's requirement list one item at a time, and make sure at least one case tests each.
- **Include all five kinds of case**: normal, boundary, invalid input, state over several steps, and time or randomness.
- **Be specific.** "Entered `abc` " is better than "bad input".
- **Write expected results first.** Then run the case.
- **Keep failures.** A row that says "Fail, then fixed" shows real testing. A plan where nothing ever failed makes a reader wonder whether you tried hard to break anything.

How to describe a failure and a fix

In the Actual result column, write what really happened. Then add what you changed. For example:

markdown

```
| 9 | Blank entry | Pressed Enter with nothing typed | Message, attempts uncha
```

Say what was wrong, how you fixed it, and that you retested.

Checking That Nothing Is Missing

Before you call a test plan finished, check it against the requirements. A simple table helps:

Requirement	Test case numbers
Ages 5 to 12 get a child ticket	3, 4
Non-numbers are rejected	8, 9, 10, 11

Ages above 120 are rejected	13, 15
Ages 65 to 120 get a senior ticket	7, 14

If a requirement has no test case, add one. If a test case supports no requirement, ask whether it is needed.

The Limits of Manual Testing

Manual testing is the right place to start, because it forces you to think about what the program should do. But it has real weaknesses:

- **It is slow.** Every change means repeating dozens of cases.
- **It is easy to skip.** When you are tired, you rerun three cases instead of thirty.
- **It is easy to get wrong.** You can misread output, or forget a step.
- **It does not scale.** A client system can have thousands of cases.

Automated testing solves this. In Week 2 you learn **pytest**, a tool that runs your test cases for you and reports which ones failed. The thinking is the same as what you practice here. Each row in your test plan, with its input and expected result, becomes one automated test. The skill you are building this week, deciding what to test and what the right answer is, is the hard part. Automating it comes next.

Common Mistakes

Mistake	**What happens**	**Fix**
Writing expected results after running	You accept whatever the program did	Write expectations first
Testing only normal cases	Bugs at the edges and in bad input go unnoticed	Add boundary and invalid cases
Vague steps such as "typed bad input"	Nobody can repeat the test	Write the exact input
Putting several checks in one case	A failure does not say which one broke	One idea per case

Testing only the cases you expect to pass	You miss real bugs	Try to break the program
Deleting failed rows	The plan hides what you learned	Keep the failure and describe the fix
Fixing a bug and rerunning only that case	An older feature may now be broken	Rerun the earlier cases too
Forgetting to reset `DEBUG_SECRET` or `TIME_LIMIT`	The finished game is broken	Add a "reset test settings" check at the end
Leaving out the line of dashes under the header row of a Markdown table	The table shows as plain text	Add the dashes line directly under the header row
Typing the pipe character inside a table cell	The cell splits in two	Describe it in words instead
Marking every row "Yes" without really running it	The plan means nothing	Run each case and look at the output

Try It Yourself

Create a file for each exercise and run it with `python3 filename.py`.

1. **Plan before code.** A program asks for a score from 0 to 100 and prints a grade: 90 and above is A, 80 to 89 is B, 70 to 79 is C, anything lower is "Try again", and anything that is not a whole number from 0 to 100 is rejected. Before you write the program, write a test plan table with at least 12 cases, including every boundary. Then write the program and run your plan.

2. **Find the boundary bug.** Write the grade program on purpose with `>` where it should say `>=` for one grade. Does your plan catch it? Which case caught it?

3. **Fix and retest.** Fix the bug from exercise 2. Update the plan the way the worked example did, with the failure noted and the old cases rerun.

4. **Test a random program.** Write a program that picks a random number from 1 to 10 and reports whether you guessed it. Add a `DEBUG_SECRET` setting. Write four test cases that depend on it.

5. **Test a timer.** Write a program that gives you 5 seconds to answer a question. Write test cases for a fast answer and a slow answer. Use the shortened limit to run them.

6. **Test state.** Write a program that lets you guess a password 3 times and counts the attempts. Write one test case that covers a sequence of bad entries followed by a good one, and say what the counter should show at the end.

7. **Pipe the inputs.** Use `printf` to run the grade program with a list of inputs from your plan. Include one run with too few inputs, and read the `EOFError`.

8. **Write it in Markdown.** Put your exercise 1 plan in a `README.md` file as a Markdown table, push it to a test repository or open the preview in PyCharm, and check that the table renders.

Why This Matters for an FDE

When you deliver software to a client, "it works on my machine" is not enough. You need to show what was tested, how, and what the results were. A clear test plan is how you earn trust: it tells a client what you checked, it tells your teammate what to rerun after a change, and it tells your future self what the code was supposed to do. The habit of asking, "What are the edges, what are the bad inputs, and what should happen?" before you write code is also how senior engineers catch problems while they are still cheap to fix.

Comprehension Check

1. What is the difference between testing and debugging?

2. Why should you write the expected result before you run a test case?

3. A rule says that a discount applies to orders of 50 dollars or more. Which three order amounts would you test at the boundary, and why?

4. Why do you rerun older test cases after you fix a bug?

5. How do `DEBUG_SECRET` and a shortened `TIME_LIMIT` make a game easier to test?

6. A test plan shows every case marked "Yes" and no failures. Why might a reader doubt it?

7. Your README table shows up as plain text with `|` characters on GitHub. What is the most likely cause?

8. Name two weaknesses of manual testing, and say what Week 2 introduces to address them.

← Previous: Using random and time, and Reading Error Messages

Next: Lab: Build Your Own Linux Environment with Docker →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!